

Constructing Temporary Tables for Real-Time Combinatorial Quiz Queries

CSCE 811 Data Modeling for Systems Development – Homework 2

Nicholas Ampimah

<ADD UNL EMAIL>

Add teammate names and emails here, if applicable

Fall 2026

Required before submission: Replace the email placeholder above and add any teammates before exporting the final submission.

Abstract

This project finds every unique combination of four quizzes for which at least P students earned a score of at least R on every quiz. Two general Python algorithms were implemented. Algorithm 1 exhaustively enumerates all $\binom{N}{K}$ quiz combinations and scans the student rows. Algorithm 2 constructs temporary combination levels and stores the intersection of qualifying student sets, pruning an intermediate combination whenever its count falls below P . Both programs infer M and N from an arbitrary input CSV and accept P , R , and K as run-time parameters. On a reproducible 30×30 dataset, both methods returned identical results. For $(P, R) = (5, 5)$, 1,141 combinations qualified and the median core times were 0.134001240 s and 0.005488753 s for Algorithms 1 and 2, respectively. For $(P, R) = (3, 7)$, 268 combinations qualified and the respective median times were 0.137743218 s and 0.002007190 s. Thus, stepwise filtering was 24.41–68.62 times faster in these tests, although both methods retain combinatorial worst-case complexity.

1 Problem Definition and Design Goals

The input is an $M \times N$ score matrix. Rows represent students, columns represent quizzes, and an entry a_{sq} is the score of student s on quiz q . For a candidate quiz set C containing K distinct quizzes, define

$$\text{count}(C) = |\{s : a_{sq} \geq R \text{ for every } q \in C\}|.$$

The required output is every unordered quiz combination C satisfying $\text{count}(C) \geq P$, together with its count. Using combinations instead of permutations ensures that, for example, $(Q1, Q2, Q3, Q4)$ and $(Q2, Q1, Q3, Q4)$ are not both reported.

The implementation goals were correctness on arbitrary compatible CSV files, no external database, flexible P , R , and K , deterministic output order, explicit validation, comparable timing, and identical output formats for both algorithms.

2 Step 1: Random Data Generation

The script `data_generate.py` uses NumPy's reproducible random-number generator. The report experiments used $M = N = 30$, integer scores from 0 through 10, and seed 811. Student IDs are S1–S30; quiz IDs are Q1–Q30. The CSV preserves the student IDs in its first column.

```

rng = np.random.default_rng(seed)
scores = rng.integers(0, 11, size=(rows, columns))
df = pd.DataFrame(scores,
    index=[f"S{i}" for i in range(1, rows + 1)],
    columns=[f"Q{i}" for i in range(1, columns + 1)])
df.index.name = "student_id"
df.to_csv(output)

```

student_id	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Q13	Q14	Q15	Q16	Q17	Q18	Q19	Q20	Q21	Q22	Q23	Q24	Q25	Q26	Q27	Q28	Q29	Q30
S1	1	8	4	6	2	1	0	10	7	6	1	0	4	0	10	5	10	5	7	7	1	5	2	3	6	8	1	5	5	10
S2	5	8	7	4	2	10	2	0	3	10	2	4	0	8	4	8	2	4	9	8	1	0	1	7	3	6	8	1	2	1
S3	2	1	8	10	10	1	8	0	7	4	4	8	1	0	0	3	3	2	6	9	2	5	8	5	4	0	1	6	3	7
S4	8	2	9	0	4	7	1	10	7	7	0	7	3	2	4	3	4	7	2	9	10	6	0	2	9	7	0	9	2	10
S5	0	4	1	7	3	8	7	5	10	7	0	5	8	2	5	0	4	6	6	8	2	7	8	6	4	7	4	0	7	10
S6	7	8	8	2	0	9	9	7	4	6	10	1	7	1	5	0	3	2	0	3	4	0	3	2	3	7	6	8	3	9
S7	6	10	0	1	4	4	1	9	5	5	8	3	1	7	7	3	10	4	10	4	4	8	2	9	1	2	7	8	3	6
S8	9	5	5	1	2	6	6	3	4	0	5	10	10	3	1	4	10	1	10	0	9	0	10	1	5	5	0	5	0	0
S9	7	7	9	3	9	7	9	2	6	2	8	4	8	7	7	9	10	1	8	9	2	6	2	9	2	9	5	6	9	8
S10	3	0	1	6	8	10	3	2	4	2	1	5	2	8	10	2	1	4	7	1	1	9	6	4	0	0	10	8	9	8
S11	10	5	1	6	10	3	4	3	6	1	2	8	3	7	3	3	6	10	3	9	10	3	8	8	2	5	5	10	3	0
S12	3	9	5	4	3	9	9	7	2	1	10	9	7	8	3	8	4	10	5	8	8	1	10	4	4	2	5	3	8	0
S13	3	9	10	3	9	3	9	3	8	6	2	4	2	7	9	0	7	4	6	7	0	8	0	3	4	0	4	6	7	4
S14	6	5	8	6	9	3	0	1	7	3	10	0	5	0	10	7	0	2	0	4	10	3	9	3	6	0	5	6	7	7
S15	3	0	0	0	0	0	2	6	9	8	1	0	3	8	8	9	5	0	0	5	8	7	10	9	1	5	3	9	0	5
S16	10	8	5	6	3	0	8	1	7	2	4	10	0	9	7	5	8	10	2	1	6	10	9	1	9	10	8	5	2	1
S17	0	1	10	9	4	4	10	3	5	9	3	7	7	6	6	5	4	0	4	0	9	5	4	7	4	1	10	3	2	1
S18	3	4	0	6	6	2	3	3	0	10	8	1	1	7	0	2	7	4	10	3	5	1	0	7	8	5	1	0	1	4
S19	0	0	0	3	0	0	6	1	7	1	7	8	0	8	3	5	0	10	0	7	2	4	0	0	4	2	3	5	1	7
S20	7	0	6	5	4	9	9	0	0	2	5	5	5	3	1	2	9	5	5	6	5	8	9	7	6	1	0	8	4	0
S21	8	1	9	10	1	4	6	4	9	7	2	2	2	1	2	5	0	5	1	1	5	4	9	7	4	7	2	7	2	2
S22	0	4	9	4	9	2	6	8	7	6	4	2	10	10	3	4	5	0	3	4	7	1	9	0	9	3	9	3	1	0
S23	6	9	7	10	2	7	8	2	5	9	5	6	2	10	5	10	1	9	4	7	2	7	9	7	0	4	4	1	2	9
S24	6	8	9	0	7	5	5	8	9	9	4	0	2	4	2	10	5	0	10	7	2	2	9	9	4	1	8	3	10	3
S25	6	0	0	5	6	2	1	4	10	5	8	6	6	1	5	10	7	10	7	1	6	7	8	9	0	6	9	10	0	0
S26	3	4	0	10	6	5	1	5	1	7	0	9	7	0	4	6	2	2	3	3	4	4	3	6	8	2	0	5	2	0
S27	7	8	1	10	4	1	7	10	0	10	8	4	2	2	5	1	6	8	5	9	1	2	2	0	4	3	8	5	2	10
S28	6	5	8	0	10	10	8	3	0	6	7	4	6	3	1	6	3	6	6	5	6	3	4	2	0	4	4	3	4	7
S29	4	7	0	10	0	1	8	7	10	5	6	5	0	0	9	0	7	5	2	1	2	7	0	2	8	1	8	1	3	0
S30	5	9	1	3	4	7	10	3	7	0	9	3	7	9	7	1	4	2	2	5	4	7	0	10	7	9	2	4	3	4

Figure 1: The complete reproducible 30×30 score dataframe generated with seed 811.

3 Step 2: Algorithm Implementations

3.1 Algorithm 1: exhaustive combination enumeration

Algorithm 1 converts the score matrix into a Boolean matrix using the threshold $a_{sq} \geq R$. It then obtains every size- K column combination with `itertools.combinations`. For each combination, it applies a row-wise logical AND across the selected columns and counts the `True` rows. Results with counts at least P are written to CSV.

```

for indices in itertools.combinations(range(len(quiz_ids)), k):
    count = int(np.all(passing[:, indices], axis=1).sum())
    if count >= p:
        results.append((tuple(quiz_ids[i] for i in indices), count))

```

This method is a direct implementation of the problem definition and provides a useful correctness baseline.

3.2 Algorithm 2: stepwise filtering with temporary structures

Algorithm 2 first creates a temporary mapping from each quiz to the set of student positions whose scores meet R . It retains only singleton quizzes with at least P students. At level d , every retained $(d-1)$ -tuple is extended only by quizzes occurring after its last quiz in the original column order. The stored student set is intersected with the new quiz's set; the new tuple is retained only if the intersection contains at

least P students. This continues from level 1 through level K :

$$(Q_i) \longrightarrow (Q_i, Q_j) \longrightarrow \cdots \longrightarrow (Q_{i_1}, \dots, Q_{i_K}).$$

```
for combination, students in level:
    last_position = order[combination[-1]]
    for next_position in range(last_position + 1, len(quiz_ids)):
        next_quiz = quiz_ids[next_position]
        common_students = students.intersection(eligible[next_quiz])
        if len(common_students) >= p:
            next_level.append((combination + (next_quiz,), common_students))
```

The pruning rule is correct because intersection is anti-monotone. If $C \subset C'$, then the students passing every quiz in C' are a subset of those passing every quiz in C :

$$S(C') = \bigcap_{q \in C'} S(q) \subseteq \bigcap_{q \in C} S(q) = S(C).$$

Therefore, once $|S(C)| < P$, no extension of C can later reach P . Extending only in increasing column order also prevents redundant permutations.

3.3 Input handling and output structure

Both programs read the first CSV column as student IDs, infer M and N , require unique student and quiz IDs, require numeric nonmissing scores, and validate $P \geq 1$ and $1 \leq K \leq N$. A value $P > M$ is allowed and correctly produces an empty result. Both output the schema

quiz_1, quiz_2, ..., quiz_K, count.

4 Step 3: Testing and Results

4.1 Reproduction instructions

From the unzipped code directory, install the two dependencies and run:

```
python -m pip install -r requirements.txt
python data_generate.py --rows 30 --columns 30 --seed 811 \
    --output raw_score_dataframe.csv

python algorithm1.py raw_score_dataframe.csv 5 5 --k 4 \
    --output algorithm1_results.csv
python algorithm2.py raw_score_dataframe.csv 5 5 --k 4 \
    --output algorithm2_results.csv

python run_experiments.py
python test_algorithms.py
```

The first positional value after the CSV is P , the second is R , and $--k$ selects the number of quizzes. The main experiment repeats each algorithm seven times and reports the median core time to reduce the effect of transient system activity. File reading and output writing are excluded from the core query timing for both algorithms. The regression test additionally compared both methods across 100 combinations of random seed, P , R , and K ; every output matched exactly.

4.2 Observed results

Table 1: Median core runtimes over seven repetitions on the seeded 30×30 dataset.

Case	P	R	Matches	Algorithm 1 (s)	Algorithm 2 (s)	Speedup
1	5	5	1,141	0.134001240	0.005488753	$24.41\times$
2	3	7	268	0.137743218	0.002007190	$68.62\times$

Table 2: Number of Algorithm 2 combinations retained at each temporary level.

Case	1-quiz	2-quiz	3-quiz	4-quiz
1: $(P, R) = (5, 5)$	30	420	1,506	1,141
2: $(P, R) = (3, 7)$	30	326	522	268

Although Case 1 has the larger P , its lower score threshold $R = 5$ leaves more students in each quiz set and therefore more intermediate combinations. Case 2 has a higher score threshold, so its student sets shrink earlier and the pruning advantage is larger.

```

Reproducible experiment console
$ python run_experiments.py
Both algorithms produced identical results for every test case.
Case 1: M=30, N=30, P=5, R=5, K=4
  matches=1141
  Algorithm 1 median=0.134001240 s
  Algorithm 2 median=0.005488753 s
  speedup=24.41x
Case 2: M=30, N=30, P=3, R=7, K=4
  matches=268
  Algorithm 1 median=0.137743218 s
  Algorithm 2 median=0.002007190 s
  speedup=68.62x

```

Figure 2: Console evidence from the reproducible two-case experiment.

```

Case 1 output CSV (P=5, R=5, K=4)

quiz_1,quiz_2,quiz_3,quiz_4,count
Q1,Q2,Q3,Q6,7
Q1,Q2,Q3,Q7,7
Q1,Q2,Q3,Q9,5
Q1,Q2,Q3,Q10,5
Q1,Q2,Q3,Q11,6
Q1,Q2,Q3,Q13,5
Q1,Q2,Q3,Q15,5
Q1,Q2,Q3,Q16,7
Q1,Q2,Q3,Q19,5
Q1,Q2,Q3,Q20,5
Q1,Q2,Q3,Q24,5
Q1,Q2,Q3,Q26,5
Q1,Q2,Q3,Q27,7
Q1,Q2,Q3,Q30,5
Q1,Q2,Q6,Q7,7
...
1141 qualifying combinations total

```

Figure 3: Beginning of the Case 1 output file. Each row is a unique ordered representation of an unordered combination.

5 Step 4: Runtime Comparison and Asymptotic Analysis

5.1 Algorithm 1

There are $\binom{N}{K}$ candidate quiz combinations. Testing one candidate requires checking K scores for each of M students, giving

$$T_1(M, N, K) = \Theta\left(MK \binom{N}{K}\right).$$

For fixed K , this is $\Theta(MN^K)$. Ignoring the required input and output storage, the temporary working space is $\Theta(MN)$ for the Boolean matrix plus $O(K)$ for one combination. If L combinations qualify, storing the output adds $\Theta(LK)$.

5.2 Algorithm 2

Creating the singleton student sets costs $\Theta(MN)$. Let A_d be the number of size- d combinations whose student sets are actually constructed. With set intersections bounded by $O(M)$, a data-sensitive bound is

$$T_2(M, N, K) = O\left(MN + M \sum_{d=2}^K A_d\right), \quad A_d \leq \binom{N}{d}.$$

In the worst case, every intermediate combination survives, so

$$T_2 = O\left(M \sum_{d=1}^K \binom{N}{d}\right),$$

which is also $O(MN^K)$ for fixed K . Thus, Algorithm 2 does not remove the combinatorial worst case. Its practical advantage is that selective thresholds make A_d much smaller than $\binom{N}{d}$ at deeper levels. Python's hash-set intersection also processes only the retained student IDs rather than rescanning the entire score row for every full combination. The implementation holds only the current and next levels, giving worst-case temporary space $O(M \max_{d \leq K} \binom{N}{d})$, in addition to input and output.

5.3 Interpretation

Algorithm 1's runtime changed little between cases because it always tests all $\binom{30}{4} = 27,405$ four-quiz combinations against all students. Algorithm 2 responded to selectivity: in Case 2, only 522 three-quiz combinations survived to generate 268 final combinations. This explains why its speedup rose from $24.41\times$ to $68.62\times$. Algorithm 1 is simpler and is a strong validation oracle for small inputs. Algorithm 2 is preferred for interactive queries when P and R prune a substantial portion of the search space.

6 Step 5: AI-Assisted Exploration

6.1 ChatGPT prompt and response

Model used: ChatGPT, GPT-5 family, accessed September 2026.

The prompt asked the model to design both exhaustive and stepwise solutions, keep all problem dimensions general, avoid an external database, and explain correctness, pruning, data structures, and complexity. Figure 4 records the prompt and response used for this work.

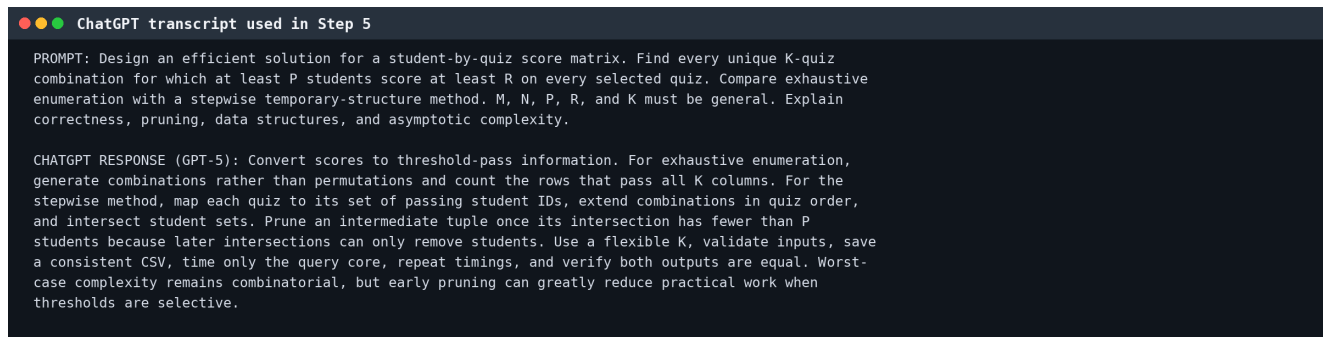


Figure 4: ChatGPT prompt and response transcript.

The most useful suggestions were: (1) use combinations rather than permutations to remove duplicates; (2) threshold the scores once; (3) represent each quiz by a qualifying-student set; (4) justify early pruning using the fact that intersections can only shrink; (5) generalize the combination size to K ; and (6) verify the optimized output against the exhaustive baseline. These recommendations are consistent with the final implementation. Repeating timings and reporting a median also made the practical comparison more defensible.

6.2 Second AI model prompt and required evidence

The exact prompt for the second model is included as `second_ai_prompt.txt`. It should be run without modification in Gemini or Grok so the comparison uses the same task statement.

Required before submission: Run `second_ai_prompt.txt` in Gemini or Grok. Insert a real screenshot below, record the model/version and access date, and replace the bracketed comparison statements with observations supported by that response. Do not submit invented AI evidence.

INSERT SECOND-AI SCREENSHOT HERE

Suggested filename: `second_ai_response.png`

Second model used: [Gemini/Grok and exact version]

Access date: [date]

Suggestions that made sense: [Summarize the second model’s useful recommendations and connect them to the implemented algorithms.]

Comparison: Both ChatGPT and [second model] recognized [verified common suggestions]. ChatGPT placed more emphasis on [verify from the responses], whereas [second model] emphasized [verify from the responses]. [State whether either response contained an incorrect, unnecessary, or especially useful suggestion.]

Preferred method/model: For the implemented solution, Algorithm 2 is preferred because it preserves exactness while pruning candidates that cannot meet P . Among the AI responses, [choose ChatGPT or the second model] was more useful because [give a concrete, response-based reason].

7 Conclusion

Both programs correctly identify all nonredundant quiz combinations meeting the student-count and score thresholds. Exhaustive enumeration is direct and useful as a reference implementation, but its work is insensitive to threshold selectivity. Stepwise filtering exploits anti-monotone student-set intersections and was 24.41–68.62 times faster on the two reproducible test cases. Its worst case remains combinatorial, yet it is the better practical choice for real-time queries when early levels can be pruned. The code is general with respect to M , N , P , R , and K , and its regression tests confirm agreement between the two methods.

A Submitted Source Files

- `data_generate.py`: reproducible CSV generator.
- `algorithm1.py`: exhaustive enumeration.
- `algorithm2.py`: general stepwise filtering from level 1 through K .
- `quiz_query_common.py`: shared validation and CSV writing.
- `run_experiments.py`: two test cases, repeated timings, and equality check.
- `test_algorithms.py`: 100 additional cross-method regression cases.
- `requirements.txt` and `README.md`: dependencies and commands.
- `second_ai_prompt.txt`: identical prompt for the required second model.